

The HexaPedal

Section HM

Phase II - 2025-10-09

Mohamad Addasi (40278616)

Fouad Elia (40273045)

Junior Boni (40287501)

Tristan Girardi (40272203)

Youssef Yassa (40265083)

Youssef Youssef (40285384)

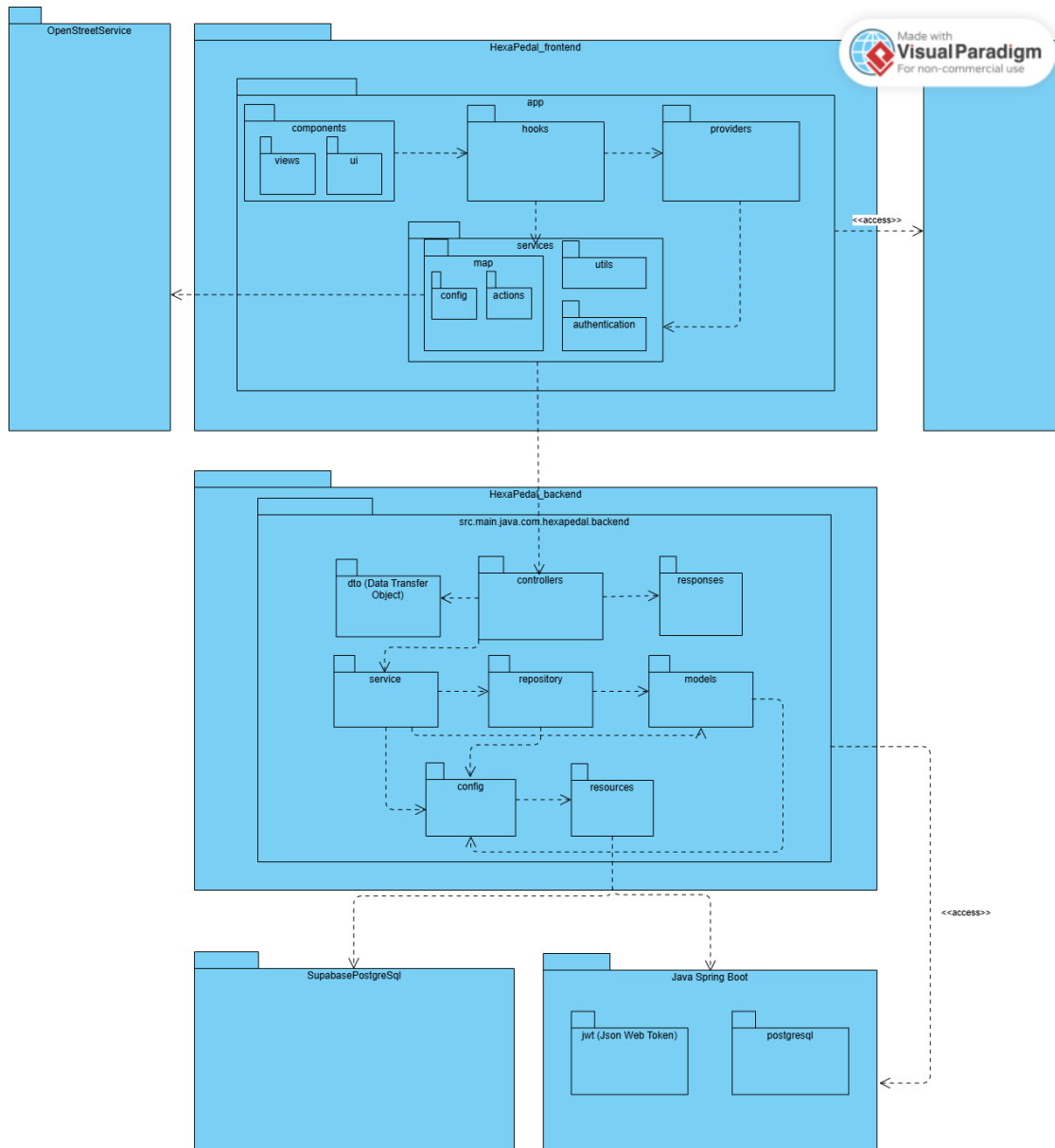
Table of Contents

Table of Contents	1
System Architecture	2
Overview	2
Presentation Layer	3
Application Layer	5
Domain Layer	6
Low-Level Infrastructure Layer	7
Technical Services Layer	8
Use Case Model	9
Use Case Diagram	10
Sequence Diagram	10
Written Use Case Model	10
Unlock Bike	11
View Map	11
View Changes	12
Reserve Bike	12
Return Bike	13
Handle Registration and Login	14
Check Bike reservation	14
Handle Payment	15
Rebalance Stations	16
Mark Bike/Station Status	16
View System Messages	17
Design pattern implementations	18
Map Repository (previously named “Map Provider” in Domain Model): Singleton	18
Map View: Observer	19
Third Design pattern	20
Contribution Self-Declaration	20

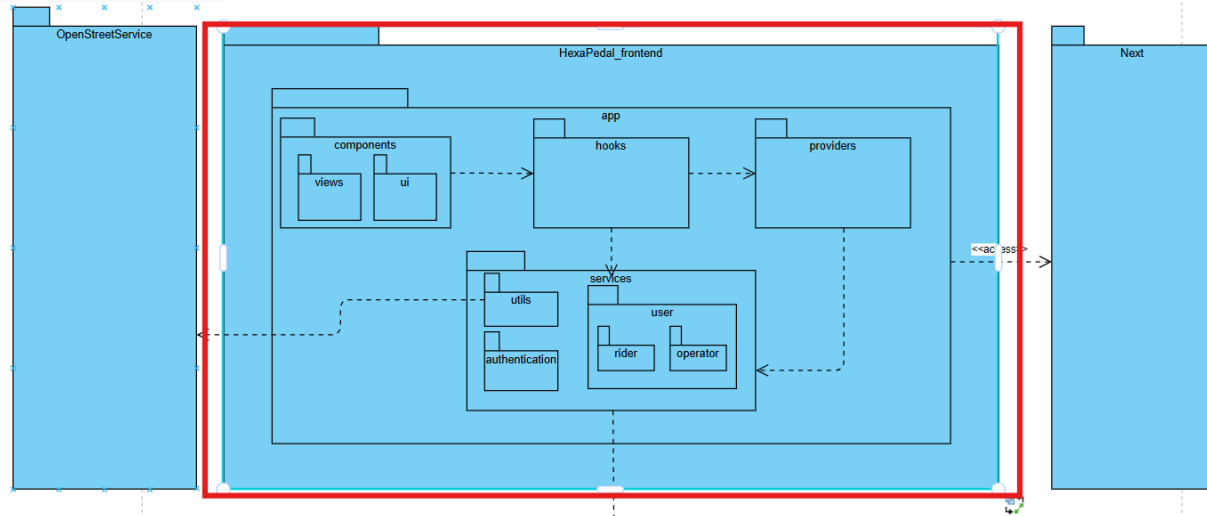
System Architecture

Our architecture is a mix of the traditional layers and the modern abilities given by frameworks such as React and Java Spring Boot. 4 of the 6 traditional layers of responsibilities are present under modern and case-specific package aliases. Nevertheless, they have similar responsibilities.

Overview



Presentation Layer



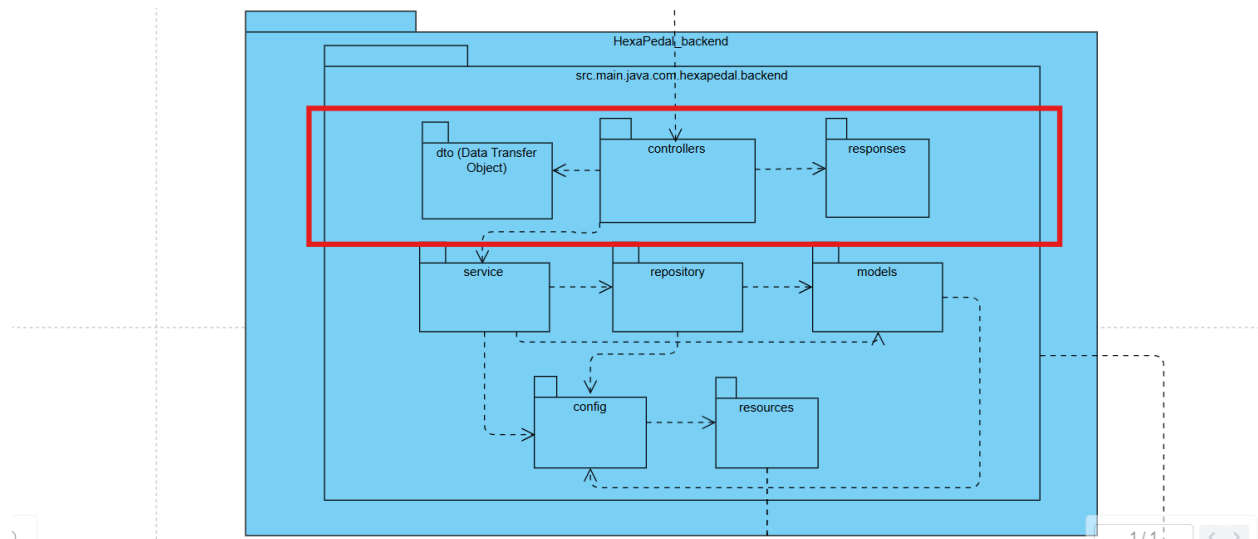
*Highlighted in **red** is the Presentation Layer*

Our presentation layer is a **closed** layer stored inside the `Hexapedal_frontend` package. This package holds the `app` package within which the meat of our frontend resides.

- **app**: Holds the different Next.js packages that allow for website routing
 - **components**:
 - **views**: Holds the main React component of each page – the core of the page. It uses components in the UI without “depending” on them.
 - **ui**: Holds the reusable components across pages such as styled buttons, styled cards, etc.
 - **hooks**: Holds our custom reusable stateful functionalities as defined within the next framework. Usually, they are named in the form of “`use____.tsx`”. They are often used as interfaces for providers.
 - **providers**: Holds global context states (i.e. stateful objects that would be required by a subset of the many components).
 - **services**: Holds functionality for triggering backend requests.
 - **utils**: Holds miscellaneous functionalities that are used for configurations or just one-time encapsulated functions impacting the whole presentation layer (i.e loading the map)

- **authentication:** Holds the triggering requests for signup and login actions.
- **user:** Holds the functionality for the use cases.
 - **rider:** Holds the functionality only related to the riders (see the use case diagram).
 - **operator:** Holds the functionality only related to the operators (see the use case diagram).

Application Layer

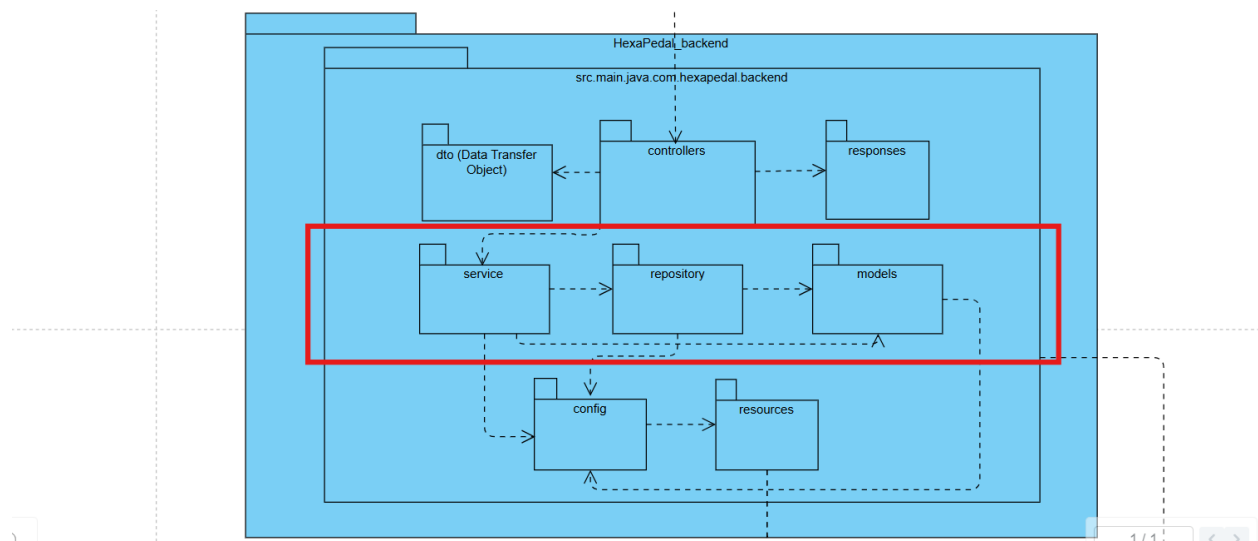


*Highlighted in **red** is the Application Layer*

Our application layer is used to orchestrate the use cases and monitor the requests made by the presentation layer. It is a **closed** layer for security and reliability, and it is the entry point of any request made to HexaPedal_backend.

- **controllers:** holds the necessary functions for invoking the appropriate logic without direct interaction with the database.
- **dto (or Data Transfer Objects):** It is the holds the "template classes" dictating what kind of input is accepted by the domain model.
- **responses:** Holds the "template classes" dictating what kind of output is sent by the domain model.

Domain Layer

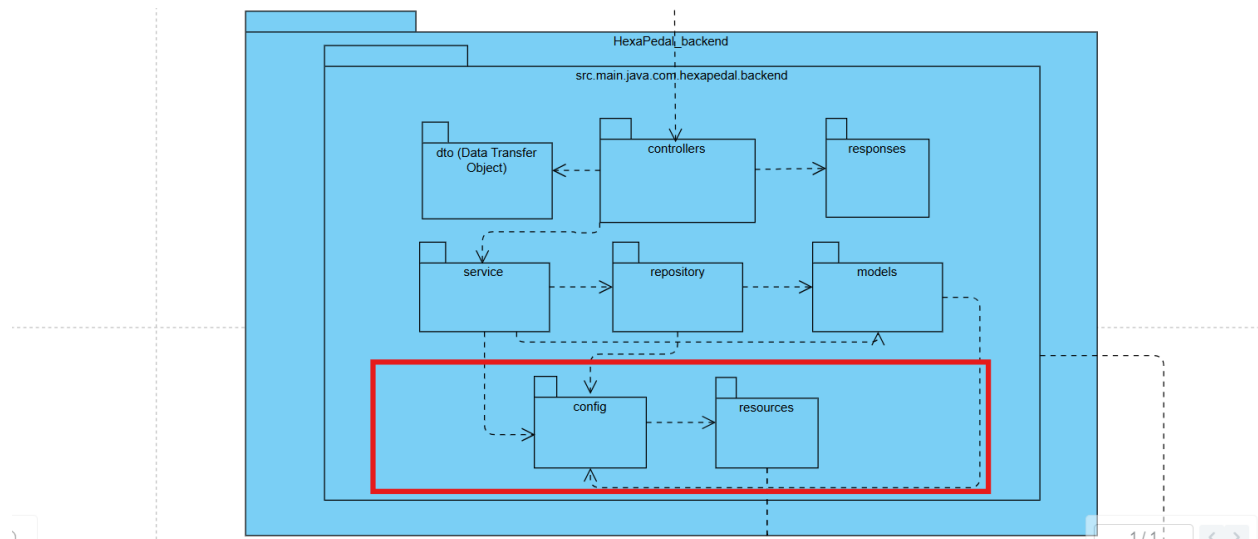


*Highlighted in **red** is the Domain Layer*

Our domain layer contains specific functions related to the bike sharing system meaning everything from the BMS, the registration module, the pricing functionality, etc. All of it will be done through this layer, hence why it is a **closed** layer.

- **service:** Holds classes with miscellaneous functionality, such as authentication service, jwt services, etc.
- **repository:** Holds classes representing table objects of specific models. Hence, these specific objects will be “experts” of the respective models that they hold, and as experts, they will be the ones that we should seek to have aggregate operations on specific model entries.
- **models:** Holds the fine-grained function. Functions like reserving a bike, unlocking a bike, and dock rebalancing will be stored in their respective classes.

Low-Level Infrastructure Layer

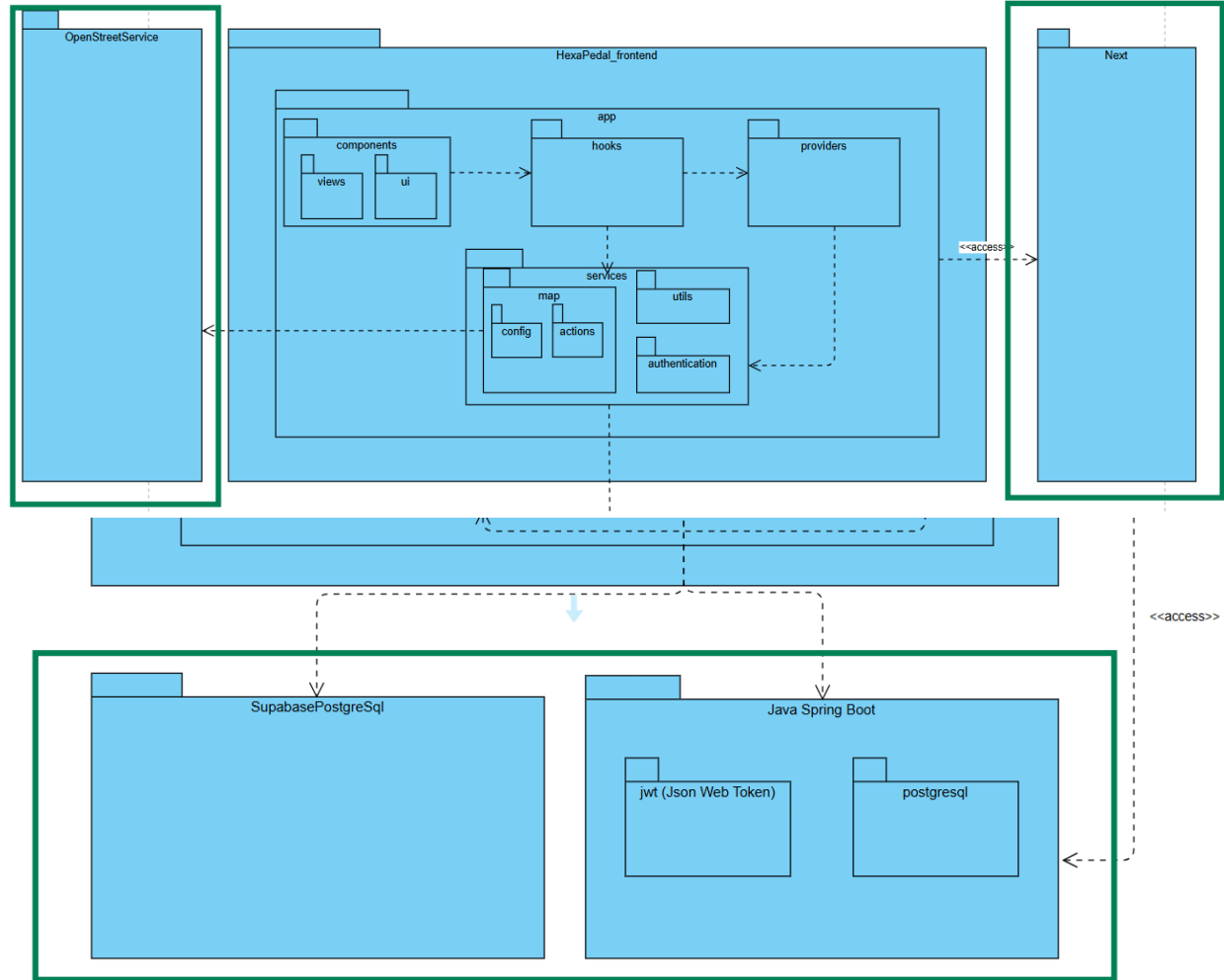


*Highlighted in **red** is the Low-Level Infrastructure Layer*

This layer is really about the configuration of the technical layer; hence, it holds low-level configuration files and resources specific to our technical services (Supabase's database, Java Spring Boot).

- **config**: Configuration for some of the external services used, such as email configuration, and security configuration.
- **resources**: Define the specific properties that will be used by external services.

Technical Services Layer

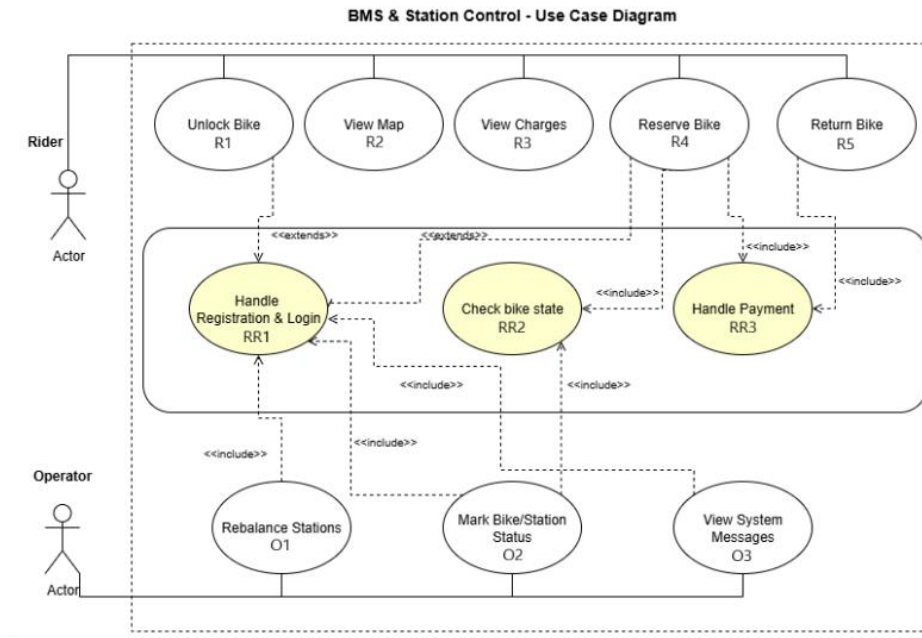


*Highlighted in **green** is the Technical Services Layer*

These are simply the external packages onto which our application depends. It is mainly influenced by our tech stack.

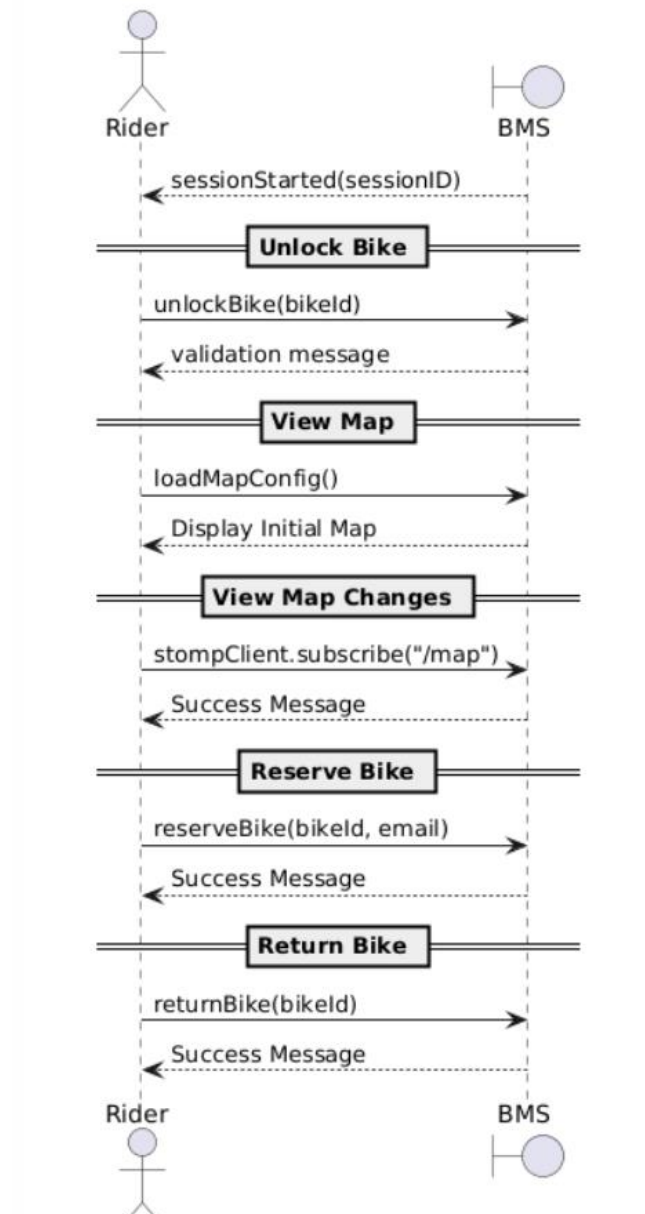
Use Case Model

Use Case Diagram



Sequence Diagram

BMS Sequence Diagram: Rider Interactions



Written Use Case Model

Unlock Bike

- **ID:** R1
- **Title:** Unlocking available bikes
- **Description:** If a user opens the application and is either a guest or is logged in with an authenticated account, then he/she can unblock a bike that is reserved under said user's name
- **Primary Actor:** Rider
- **Preconditions:**
 - Bike is reserved by the current user
 - User is a rider (guest or logged in)
 - Dock is not empty
- **Postconditions:**
 - Bike state turns to on_trip
 - Update the dock state from "occupied" to "empty"
 - Create an event object recording the bike's change of state
 - Update the number of bikes docked in the dock station
 - Update the list of bikes docked in the dock station
- **Inputs:**
 - The bike Id
- **Outputs:**
 - Operation Log (event object) confirming successful/unsuccessful unlock
- **Main Success Scenario:**
 - User is a rider
 - Bike is reserved by user
 - User unlocks bike
 - Bike state is now on_trip

View Map

- **ID:** R2
- **Title:** View Map
- **Description:** When opening the application, the users, whether they are operators or riders, can see the Montreal map with the positions of dock stations highlighted and simplified labels.
- **Primary Actor:** User
- **Preconditions:**
 - Configuration file has been successfully loaded

- Map API provider is available
- **Postconditions:**
 - Station Markers, Docks Stations and Bikes have been created
- **Inputs:**
 - Configuration file
- **Outputs:**
 - The Map UI is displayed to the user
- **Main Success Scenario:**
 - Users open the app
 - BMS loads the configuration file
 - The dock stations, station markers and the bikes information are loaded
 - The user sees the map with the relevant information

View [Map] Changes

- **ID:** R3
- **Title:** View the bike and docker stations attributes change in real time
- **Description:** When the user accesses the application and other users make changes to some of the bikes, he/she will see these changes in real time.
- **Primary Actor:** User
- **Preconditions:**
 - Concurrent users are actively changing the real time attributes of bikes and dock stations
 - Users are subscribed to the proper
- **Postconditions:**
 - The attributes of each bike and dock station are updated in real time
- **Inputs:**
 - Signal of event creation
- **Outputs:**
 - Updated view of the bikes and dock stations
- **Main Success Scenario:**
 - User accesses the app
 - Other users make some changes to bikes and/or dock stations
 - User sees the changes in real time

Reserve Bike

- **ID:** R4
- **Title:** Reserve an available bike

- **Description:** Users that are riders can decide to reserve an available bike for up to 5 minutes and they have to pay a reservation fee.
- **Primary Actor:** Rider
- **Preconditions:**
 - Bike has state “available”
 - User is a rider
- **Postconditions:**
 - Bike’s state is turned into “reserved”
 - Bike’s expiration date and time is updated
 - Event is created recording the Bike’s change of state
- **Inputs:**
 - Bike’s Id
- **Outputs:**
 - Request for payment information
 - Operation log (event object) confirming the successful/unsuccessful change of bike’s state
- **Main Success Scenario:**
 - User is a rider
 - User selects a bike with state “available” through the user interface
 - The bike now has state “reserved” for the next 5 minutes

Return Bike

- **ID:** R5
- **Title:** Return bike that was previously “on trip” to an available dock station
- **Description:** A rider that has finished his/her trip will return said bike to a dock station with at least one dock available.
- **Primary Actor:** Rider
- **Preconditions:**
 - User is a rider
 - Bike is “on trip”
 - Dock station has at least one empty dock
 - Dock station is not “out of service”
- **Postconditions:**
 - Creation of event object recording the bike’s change of state
 - Bike’s state turns to “available”
 - Dock’s state turns from “empty” to “occupied”
 - Update the number of bikes docked in the dock station
 - Update the list of bikes docked in the dock station

- **Inputs:**
 - The bike Id
 - Dock station position
- **Outputs:**
 - Operation log (event object) confirming a successful/unsuccessful return of bike
- **Main Success Scenario:**
 - The user completes his/her trip
 - The user finds an available dock station
 - The user returns the bike and occupies a dock
 - The bike's state change from "on trip" to "available"

Handle Registration and Login

- **ID:** RR1
- **Title:** Handles the registration
- **Description:** The registration and login system recognize the authentication and authorization of every user's request.
- **Primary Actor:** Registration and Login Management System
- **Preconditions:**
 - The operator users have predetermined credentials
 - Riders signed up for our app through the user interface
- **Postconditions:**
 - Event log records user login activity
 - Modifies the actions available to the user
- **Inputs:**
 - User credentials
- **Outputs:**
 - Authenticated user data
 - Operation log (event object) recording a successful/unsuccessful authentication attempt
- **Main Success Scenario:**
 - User registers
 - User successfully login with created credentials or operational user credentials

Check Bike reservation

- **ID:** RR2
- **Title:** Check bike's reservation
- **Description:** The registration and login system
- **Primary Actor:** Registration and Login Management System
- **Preconditions:**
 - o The bike instance exists
 - o The user is a rider
- **Postconditions:**
 - o No state change
- **Inputs:**
 - o The bike's id
- **Outputs:**
 - o Whether the bike is in status "reserve" or not
- **Main Success Scenario:**
 - o User asks for bike's reservation status
 - o The registration and login management system checks confirms that the user has the right to access such information, and sends him/her the bike's status

Handle Payment

- **ID:** RR3
- **Title:** Handle Payment information
- **Description:** The registration module takes care of providing the relevant payment information of the rider user to the pricing by billing module.
- **Primary Actor:** Registration and Login Management System
- **Preconditions:**
 - o User is a rider
 - o Rider filled the relevant payment information, and it is recorded in their corresponding payment info object
 - o Rider sent a valid request to a paid service
- **Postconditions:**
 - o Creation of a billing object
 - o Billing object is associated with the rider
 - o Billing is filled with Payment Info
 - o Create an event object recording the payment request

- **Inputs:**
 - Service requested
- **Outputs:**
 - Operation log (event object) recording the payment request
- **Main Success Scenario:**
 - User filled their payment info
 - User requests for a paid service (bike reservation or usage plan)
 - User is billed based on the requested service with the appropriate payment information

Rebalance Stations

- **ID:** O1
- **Title:** Rebalance dock stations' bikes docked
- **Description:** To deal
- **Primary Actor:** Operators can update the number of bikes at a dock station. The operator can decide to remove or add some bikes to the dock station.
- **Preconditions:**
 - User is an operator
 - Dock station is not "out of service"
 - Dock station has empty docks
- **Postconditions:**
 - Update the list of docked bikes
 - Update the number of bikes docked
 - Creation of an event object recording the change of attributes in the docking station
- **Inputs:**
 - Dock station position
 - Operation flag to distinguish adding or removing bikes from the dock station
 - List of bike objects to add or remove
- **Outputs:**
 - Operation log (event object) confirming the successful/unsuccessful change in the dock station state
- **Main Success Scenario:**
 - User is authenticated as an operator
 - Operator sees on the map that a dock station is lacking in bikes docked
 - Physically, the operator moves the truck to another station

- Through the app's UI, the operator manually removes a bike from one station, and adds the bike to another docking station

Mark Bike/Station Status

- **ID:** O2
- **Title:** Update Bike/Station Status
- **Description:** Depending on the physical state of the bike or dock station, the operator can change its virtual status.
- **Primary Actor:** Operator
- **Preconditions:**
 - The user is an operator
 - The object is either an existing bike or a dock station instance
 - If the object is a bike, its current state is "available"
 - If the object is a dock station, its current state is not "out of service"
- **Postconditions:**
 - The object's status is updated
 - Creation of an event object recording the object's state change
- **Inputs:**
 - The bike instance (can be null)
 - The dock station instance (can be null)
- **Outputs:**
 - An operation log (event object) confirming successful/unsuccessful update of state change of the given object
- **Main Success Scenario:**
 - After physical inspections, the operator deems the bike to be defective
 - Through the UI, the operator manually changes the state of the bike from "available" to "out of service"

View System Messages

- **ID:** O3
- **Title:** View real time system logs
- **Description:** The operator can consult the operator logs that describe the current state of the system.
- **Primary Actor:** Operator
- **Preconditions:**

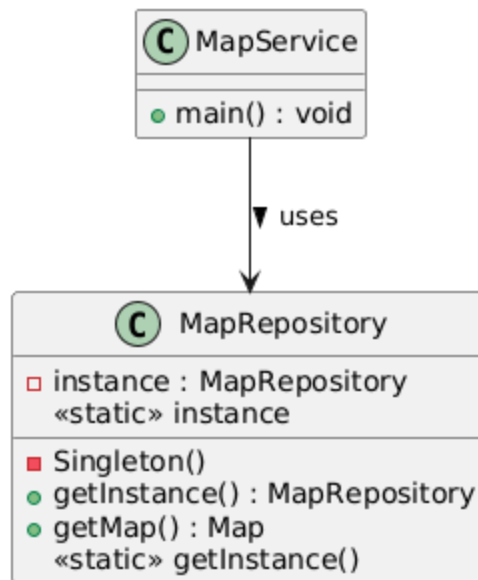
- User is an operator
- User has access to the dashboard view
- Non-operational users triggered events
- **Postconditions:**
 - No change in the system's state
- **Inputs:**
 - None
- **Outputs:**
 - List of operator logs (events) in a chronological order
- **Main Success Scenario:**
 - User authenticates as an operator
 - User looks at the dashboard
 - User sees a sequence of logs about various events

Design pattern implementations

Map Repository (previously named “Map Provider” in Domain Model): Singleton

For our implementation of the map object, we need to use a map provider which would load the initial station markers (and the available bikes in the system). Since it is going to be a model required to fetch and share such a heavy transaction over the network, having a single instance responsible for making the call would be greatly beneficial as it would open the opportunity of caching stale data server side. Having only one instance of an object serving as a critical interface object between the server and the application makes the codebase very expandable, considering that extensibility is an important factor of this project as we are going iterations by iterations.

Map Provider Singleton Design Pattern

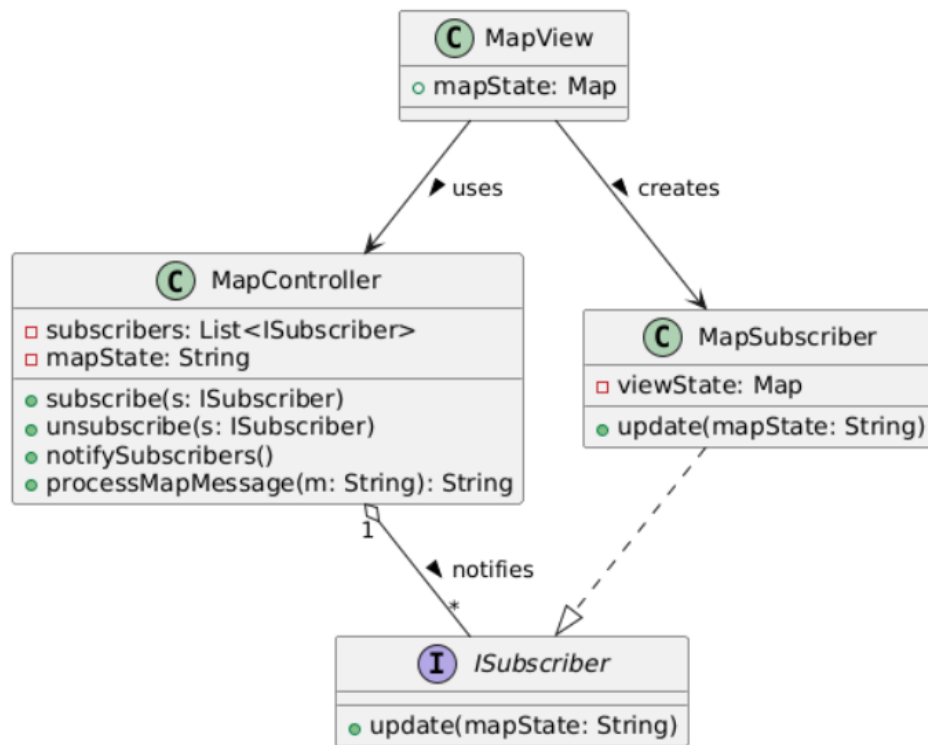


Link to [png](#)

Map View: Observer

Another important function of the map in our overall application is that it is the main display for our users. It is through the map that riders would be able to interact with the bikes, and it is also through that interface that operators would be able to monitor other features of the app; hence, it is important for the map to always display the most up to date information. Designing the relationship between the Map Repository (backend) and the numerous Map Views as an Observer and a Publisher is therefore a must. The map repository is thus responsible for constantly publishing its latest updates to the subscribers. Note that Spring Boot has some predefined infrastructure that would facilitate this communication from backend to frontend. Despite the infrastructure put in place, we still must design the publisher and subscriber design pattern as shown below.

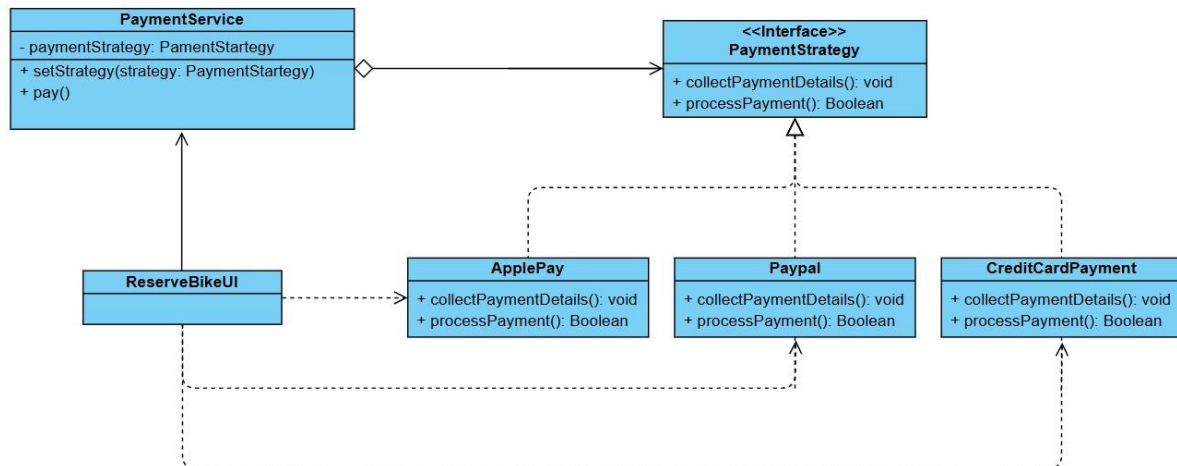
Map View Observer Design Pattern (Spring + Stateful View)



Link to [png](#)

Payment: Strategy

Another important function of our application is the ability to process different types of payments efficiently and flexibly. We want to design our payment system using a Strategy architecture so that we can easily integrate various payment options such as PayPal and credit cards. This design allows us to define a family of payment algorithms, encapsulate each one, and make them interchangeable without modifying the core payment logic. By doing so, we gain the flexibility to extend our payment functionalities in the future — for instance, by adding support for mobile wallets or cryptocurrency — without having to refactor existing code. The application can dynamically select the appropriate payment strategy at runtime based on the user's chosen method. Implementing this Strategy design pattern ensures that our payment processing remains modular, maintainable, and open for extension while being closed for modification.



Contribution Self-Declaration

All contributing members have contributed equally to this Phase delivery